

# Тема урока: свойства, функция property()

1. Свойства
2. Функция `property()`
3. Объект `property`

**Аннотация.** Урок посвящен свойствам и функции `property()`.

## Свойства, функция property()

Уроком ранее мы выяснили, что защищенные и приватные атрибуты в Python на самом деле не являются скрытыми. Помимо этого мы познакомились с концепцией геттеров и сеттеров, которая позволяет избегать прямого доступа к атрибутам, а также добавлять дополнительную логику при изменении их значений.

Рассмотрим следующее определение класса `Cat`:

```
class Cat:
    def __init__(self, name):
        self._name = name                # имя кошки

    def get_name(self):
        # геттер, используется для
        # получения имени
        return self._name

    def set_name(self, name):
        # сеттер, используется для
        # изменения имени
        if isinstance(name, str) and name.isalpha():
            self._name = name
        else:
            raise ValueError('Некорректное имя')
```

Экземпляр данного класса содержит атрибут `_name` и два метода `get_name()` и `set_name()`, позволяющих работать с этим атрибутом. Если мы решим добавить еще один атрибут экземпляра нашего класса, нам снова потребуется определить для него два метода — геттер и сеттер.

Несмотря на то что геттеры и сеттеры даже в таком виде выполняют свою основную задачу, работать с атрибутами с помощью них становится несколько сложнее, так как атрибуты перестают быть атрибутами как таковыми — мы обращаемся к ним и изменяем их значения с помощью методов. Более того, для каждого атрибута нужно помнить оба его метода.

Самый популярный способ упростить работу с атрибутами, не потеряв все преимущества геттеров и сеттеров, — превратить их в **свойства**. Свойства предоставляют промежуточную функциональность между атрибутами и методами. Другими словами, они позволяют создавать методы, которые ведут себя как атрибуты.

В Python свойства определяются как атрибуты классов, за которыми закрепляются соответствующие геттеры, сеттеры и делитеры. Так как свойства являются атрибутами класса, они доступны всем экземплярам этого класса.

Для создания свойств используется встроенная функция `property()`. Она принимает четыре аргумента:

- `fget` — функция для получения значения атрибута (геттер)
- `fset` — функция для установки значения атрибута (сеттер)
- `fdel` — функция для удаления атрибута (делитер)
- `doc` — строка документации

Функция `property()` возвращает специальный объект `property` — свойство на основе переданных геттера, сеттера и делитера.



Все четыре аргумента функции `property()` являются необязательными и по умолчанию имеют значение `None`.

Приведенный ниже код определяет свойство `name`, за которым закреплены геттер `get_name()` и сеттер `set_name()`:

```
class Cat:
    def __init__(self, name):
        self._name = name

    def get_name(self):
        return self._name

    def set_name(self, name):
        if isinstance(name, str) and name.isalpha():
            self._name = name
        else:
            raise ValueError('Некорректное имя')

    name = property(get_name, set_name) # создаем свойство name для
    # управления именем (атрибутом _name)
```

Теперь работать с атрибутом `_name` можно через свойство `name`. При этом во время обращения к свойству `name` как к атрибуту (`self.name`) будет неявно вызываться метод `get_name()`, а во время установки значения свойству `name` как атрибуту (`self.name = value`) будет неявно вызываться метод `set_name()`, которому в качестве аргумента будет передано значение `value`.

Приведенный ниже код:

```
class Cat:
    def __init__(self, name):
        self._name = name

    def get_name(self):
        return self._name

    def set_name(self, name):
        if isinstance(name, str) and name.isalpha():
            self._name = name
        else:
            raise ValueError('Некорректное имя')

    name = property(get_name, set_name)

cat1 = Cat('Кемаль')
cat2 = Cat('Роджер')

print(cat1.name)                # равнозначно
cat1.get_name()
print(cat2.name)                # равнозначно
cat2.get_name()
```

ВЫВОДИТ:

```
Кемаль
Роджер
```

Чтобы убедиться в том, что обращение к свойству `name` неявно вызывает метод `get_name()`, мы можем добавить в данный метод вывод сообщения.

Приведенный ниже код:

```
class Cat:
    def __init__(self, name):
        self._name = name

    def get_name(self):
        print(f'Возвращаю имя {self._name}')
        return self._name

    def set_name(self, name):
        if isinstance(name, str) and name.isalpha():
            self._name = name
        else:
            raise ValueError('Некорректное имя')

    name = property(get_name, set_name)

cat1 = Cat('Кемаль')
cat2 = Cat('Роджер')

print(cat1.name)
print(cat2.name)
```

ВЫВОДИТ:

```
Возвращаю имя Кемаль
Кемаль
Возвращаю имя Роджер
Роджер
```

Как уже было сказано выше, при установке значения свойству `name` неявно вызывается метод `set_name()`.

Приведенный ниже код:

```
class Cat:
    def __init__(self, name):
        self._name = name

    def get_name(self):
        return self._name

    def set_name(self, name):
        if isinstance(name, str) and name.isalpha():
            self._name = name
        else:
            raise ValueError('Некорректное имя')

    name = property(get_name, set_name)

cat1 = Cat('Кемаль')
cat2 = Cat('Роджер')

cat1.name = 'Рэтчет' # равнозначно
cat1.set_name('Рэтчет')

print(cat1.name)
print(cat2.name)
```

ВЫВОДИТ:

```
Рэтчет
Роджер
```

Чтобы убедиться в том, что установка значения свойства `name` неявно вызывает метод `set_name()`, мы можем попытаться установить некорректное имя.

Приведенный ниже код:

```
class Cat:
    def __init__(self, name):
        self._name = name

    def get_name(self):
        return self._name

    def set_name(self, name):
        if isinstance(name, str) and name.isalpha():
            self._name = name
        else:
            raise ValueError('Некорректное имя')

    name = property(get_name, set_name)

cat = Cat('Кемаль')

cat.name = 1 # равнозначно
cat.set_name(1)
```

приводит к возбуждению исключения:

```
ValueError: Некорректное имя
```

Поскольку все аргументы функции `property()` являются необязательными, мы можем определять свойства, которые, например, доступны только для чтения. При попытке изменить значение такого свойства будет возбуждено исключение.

Приведенный ниже код:

```
class Cat:
    def __init__(self, name):
        self._name = name

    def get_name(self):
        return self._name

    name = property(get_name)

cat = Cat('Кемаль')

cat.name = 'Роджер'
```

приводит к возбуждению исключения:

```
AttributeError: property 'name' of 'Cat' object has no setter
```

## Вызов сеттера при создании экземпляра

Свойства позволяют проверять на корректность значения перед их заменой. Однако если в инициализаторе у нас нет должной проверки, экземпляр класса все равно можно будет создать некорректными данными.

Приведенный ниже код:

```
class Cat:
    def __init__(self, name):
        self._name = name                # прямое обращение к атрибуту
    _name

    def get_name(self):
        return self._name

    def set_name(self, name):
        if isinstance(name, str) and name.isalpha():
            self._name = name
        else:
            raise ValueError('Некорректное имя')

    name = property(get_name, set_name)

cat = Cat(-1)                           # устанавливаем некорректное
имя при создании

print(cat.name)
```

ВЫВОДИТ:

-1

Решить данную проблему можно добавлением необходимой проверки в инициализатор.

Приведенный ниже код:

```
class Cat:
    def __init__(self, name):
        if isinstance(name, str) and name.isalpha():
            self._name = name
        else:
            raise ValueError('Некорректное имя')

    def get_name(self):
        return self._name

    def set_name(self, name):
        if isinstance(name, str) and name.isalpha():
            self._name = name
        else:
            raise ValueError('Некорректное имя')

    name = property(get_name, set_name)

cat = Cat(-1)

print(cat.name)
```

приводит к возбуждению исключения:

```
ValueError: Некорректное имя
```

Несложно заметить, что мы дублируем немалый объем кода, добавляя проверку как в сеттер, так и в инициализатор. Вместо этого гораздо удобнее было бы воспользоваться уже реализованной в сеттере проверкой. И мы можем сделать это, если воспользуемся свойством прямо в инициализаторе.

Приведенный ниже код:

```
class Cat:
    def __init__(self, name):
        self.name = name                                # обращение к свойству name

    def get_name(self):
        return self._name

    def set_name(self, name):
        if isinstance(name, str) and name.isalpha():
            self._name = name
        else:
            raise ValueError('Некорректное имя')

    name = property(get_name, set_name)

cat = Cat(-1)

print(cat.name)
```



приводит к возбуждению исключения:

```
ValueError: Некорректное имя
```

Обратите внимание, что в инициализаторе мы не создаем атрибут `name`, мы обращаемся к уже имеющемуся свойству `name` и изменяем его значение, что приводит к вызову сеттера, внутри которого происходит создание атрибута `_name` с соответствующим значением.

Атрибут и соответствующее свойство обычно имеют схожие имена. Атрибут, как правило, является защищенным, а свойство, которое управляет этим атрибутом, публичным. Поэтому разница заключается только в наличии символа нижнего подчеркивания ( `_` ) в начале имени **атрибута**. Это важное отличие, которое следует четко понимать при работе со свойствами чтобы избежать ошибок.

## Объект property

Свойства являются атрибутами классов, которые доступны всем экземплярам этих классов. Обращаясь к свойству через экземпляр класса, мы работаем с ним как с атрибутом, неявно вызывая соответствующие методы (геттер, сеттер и делитер). При этом само свойство является обычным объектом, и мы можем получить к нему доступ через класс, в котором это свойство определено.

Приведенный ниже код:

```
class Cat:
    def __init__(self, name):
        self._name = name

    def get_name(self):
        return self._name

    def set_name(self, name):
        if isinstance(name, str) and name.isalpha():
            self._name = name
        else:
            raise ValueError('Некорректное имя')

    name = property(get_name, set_name)

print(Cat.name)
print(type(Cat.name))
```

выводит (адрес может отличаться):

```
<property object at 0x000001829C8F56C0>
<class 'property'>
```

Таким образом, свойство — это атрибут класса, который управляет атрибутами экземпляров. Мы также можем считать свойство набором методов (геттер, сеттер, делитер), собранных вместе в единый объект. Обратиться к методам, содержащимся в свойстве, можно через атрибуты `fget`, `fset` и `fdel`.

Приведенный ниже код:

```
class Cat:
    def __init__(self, name):
        self._name = name

    def get_name(self):
        return self._name

    def set_name(self, name):
        if isinstance(name, str) and name.isalpha():
            self._name = name
        else:
            raise ValueError('Некорректное имя')

    name = property(get_name, set_name)

print(Cat.name.fget)           # обращаемся к геттеру
свойства
print(Cat.name.fset)           # обращаемся к сеттеру
свойства
print(Cat.name.fdel)           # обращаемся к делитеру
свойства
```

выводит (адрес может отличаться):

```
<function Cat.get_name at 0x000001F8C2FC1090>
<function Cat.set_name at 0x000001F8C2FC1120>
None
```

## Примечания

**Примечание 1.** `property` — это класс, предназначенный для работы как функция, а не как обычный класс, поэтому большинство разработчиков называют ее функцией. По этой же причине `property()` не соответствует соглашению Python об именовании классов.

**Примечание 2.** Значения, возвращаемые свойствами, необязательно должны быть фиксированными, они могут вычисляться при каждом обращении к ним.

Приведенный ниже код:

```
class Cat:
    def __init__(self, breed, name):
        self.breed = breed
        self.name = name

    def get_info(self):
        return self.breed + ' ' + self.name

    info = property(get_info)

cat = Cat('Британский', 'Кемаль')

print(cat.info)
```

ВЫВОДИТ:

Британский Кемаль

Если вычисление значения довольно затратно, мы можем вычислить его единожды и сохранить возвращая при последующих вызовах сохраненное ранее значение.

Приведенный ниже код:

```
class Cat:
    def __init__(self, breed, name):
        self.breed = breed
        self.name = name
        self._info = None

    def get_info(self):
        if self._info is None:
            self._info = self.breed + ' ' + self.name
        return self._info

    info = property(get_info)

cat = Cat('Британский', 'Кемаль')

print(cat.info)
```

ВЫВОДИТ:

Британский Кемаль

**Примечание 3.** Если значение свойства, вычисляемого на основе различных атрибутов, можно изменять, нужно не забывать изменять и значения всех этих атрибутов.

Приведенный ниже код:

```
class Cat:
    def __init__(self, breed, name):
        self.breed = breed
        self.name = name

    def get_info(self):
        return self.breed + ' ' + self.name
```

```
def get_info(self, info):
```

